



CSCI 152 Performance and Data Structures

Exercise 1

Deadline #1: June 14, 2025 at 11.59 PM

1 Value and Reference Semantics, Dynamic Memory

The purpose of this exercise is to practice writing C++ code and understanding the difference between value and reference semantics. For this exercise, you are asked to implement a class `csci152_student` to represent students who are taking 152, which calculates their final grades based on assignment scores, quizzes, and the final exam.

Before you begin, it is important that you carefully read our class policies regarding programming assignments and lab exercises [here](#) and [here](#).

No need to do the live grading for this exercise.

2 Tasks

1. The fields and interface for this class are provided in the header file `csci152_student.h` — you should not edit this file! Your job is to create a separate implementation file `csci152_student.cpp`, which provides bodies for the constructors, functions, and the destructor of the class following the descriptions (and hints) and invariants given in the header. To help you out, we have also provided a simple testing file `main.cpp` and `Makefile`, but be sure to come up with more thorough tests.
2. This class should work for different numbers of quizzes and assignments, using the grading rubric – described in `csci152_student.h`. This assignment requires that you do some memory management – notice that there are two pointer fields in the class for referencing variable-length arrays. We cannot use fixed-length arrays for this, since we need to handle different cases where we have a different number of quizzes and assignments. Because of this complication, you really should run Valgrind on your own testing class and `csci152_student` class implementation before submitting.
3. To make things a little easier, you don't have to worry about cases where a user might provide bad inputs, for example, scores that are out of the specified range. This approach is considered strict, old-school Design by Contract. If a user provides bad inputs, it is their own fault when things go bad!
4. You may also notice the use of the type `std::initializer_list` in the main constructor to take in the assignment and quiz scores as parameters. Initializer lists in C++ are somewhat like arrays, but they allow us to use convenient syntax such as `{75, 88, 92, 81, 100}` to directly provide a list of constants when we construct new objects. You can see this in the provided `main.cpp` when we want to simply list the assignment and quiz scores without declaring additional variables:

```
csci152_student agood("Agood Stutante",
                    {100, 100, 100, 100},
                    {20.0, 20.0, 20.0, 2.3, 20.0, 20.0},
                    100.0
                    );
```

5. You will need to use and access the items in these initializer lists in the main constructor implementation. It will be sufficient to use the `ilist.size()` function to get the number of items in the initializer list, and to iterate through the items, you can use the following loop structure:

```
for(auto item: ilist) {
    // code that does something with the item
}
```



Note that we are using the keyword `auto` here in place of the explicit type of the items in the `ilist` (unsigned `int` or `double`). It is fine to use the explicit types here instead, but just know that the compiler can directly infer the item type from the iterator list.

3 Submit

For this exercise, upload the following files in Moodle under "Exercise 1" by the deadline:

1. `csci152_student.cpp`.

The name and file extensions of these files must be exact, and do not archive them. The class names along with their function signatures must be exact. In the case of these errors, they will not be recognized by our testing system, and they will get a score of 0.

4 Questions?

If you have questions about the exercise, please ask during lab sessions or post your questions on [Piazza](#). However, never post your code publicly – post it "private to instructors" in Piazza instead.